ITA06 – MACHINE LEARNING

ASSIGNMENT 2 – CO2

Neural Network Representation, Perceptrons, Backpropagation, Deep Learning and Optimization

Student details	
Name	Mohan S
Register Number	192421424
Department	B. Tech IT
Course name and faculty name	ITA0605– Machine Learning and Dr. Revathi

Q1 – Design A Neural Network Representation For A Classification Problem

Input representation and preprocessing

The input vector is $x = [x1, x2, x3]^T$, where x1 is surface roughness, x2 is dimensional deviation and x3 is vibration level. The target y is 0 for Acceptable and 1 for Defective.

All three measurements are numerical. Fit a standard scaler on the training set only: $x'j = (xj − μj)/σj$. Scaling is required because measurements may use different units and ranges. It prevents a large-scale feature from dominating gradient updates. Missing values should be imputed using training-set statistics, and extreme sensor errors should be checked.

Candidate architecture 1: no hidden layer

Diagram: [x1, x2, x3] → [single sigmoid output ŷ]. The network is a logistic-regression classifier.

Input neurons = 3; output neurons = 1; hidden neurons = 0. The output activation is sigmoid, $σ(z)=1/(1+e^{−z})$. Predict Defective when $ŷ ≥ 0.5$; otherwise predict Acceptable.

Computation: $z = w1x1 + w2x2 + w3x3 + b$; $ŷ = σ(z)$. Trainable parameters = 3 weights + 1 bias = 4.

Candidate architecture 2: one hidden layer

Diagram: [x1, x2, x3] → [h1, h2, h3, h4] → [ŷ]. Use four ReLU hidden neurons and one sigmoid output neuron.

Hidden computation: $h = ReLU(W1x + b1)$; output: $ŷ = σ(W2h + b2)$. Input neurons = 3, hidden neurons = 4, output neurons = 1. Parameter count = (3×4)+4 + (4×1)+1 = 21 trainable weights and biases.

Worked forward pass for architecture 1

Use scaled input x = [0.20, 0.40, 0.60], weights w = [0.50, −0.40, 0.30] and bias b = −0.10.

z = (0.50×0.20) + (−0.40×0.40) + (0.30×0.60) − 0.10 = 0.10 − 0.16 + 0.18 − 0.10 = 0.02.

$\hat{y} = \sigma(0.02) = 1/(1+e^{-0.02}) = 0.5050$. Since $0.5050 \geq 0.5$, the predicted class is Defective (1).

Architecture recommendation

If defectiveness depends only on an additive linear combination of the three measurements, the no-hidden-layer model is simpler, faster and easier to interpret. If combinations or interactions matter (for example, moderate roughness becomes dangerous only when vibration is also high), a hidden layer is appropriate because it can form nonlinear feature combinations.

Evidence for a hidden layer should include a validation-set improvement over the linear model, stable results across several random seeds, a nonlinear pattern in plotted data or residual errors, and acceptable model complexity. A more complex architecture should not be selected solely because it achieves higher training accuracy.

Architecture	Layer sizes	Activation	Parameters
No hidden layer	$3 \rightarrow 1$	Sigmoid output	4
One hidden layer	$3 \rightarrow 4 \rightarrow 1$	ReLU hidden; sigmoid output	21

Q2 – Investigate the Learning Behaviour and Limitations of a Perceptron

Original dispatch rule: AND

Dispatch is permitted only when Packaging Complete=1 AND Quality Check Passed=1. The target is $y=x1 \wedge x2$.

Packaging x1	Quality x2	Target y
0	0	0
0	1	0
1	0	0
1	1	1

A single perceptron can learn this rule because the positive point (1,1) can be separated from the three negative points by a straight line. For example, $x1+x2-1.5=0$ separates them.

Configuration: initial weights w1=0, w2=0; bias $b=-0.2$; learning rate $\eta=0.1$; threshold 0; output 1 if $net \geq 0$, otherwise 0; maximum 20 epochs. Training order is (0,0), (0,1), (1,0), (1,1).

Before training, net values are −0.2, −0.2, −0.2 and −0.2, so predictions are 0,0,0,0. Only the final row is misclassified. Update rule: $w_i \leftarrow w_i + \eta(y-\hat{y})x_i$; $b \leftarrow b + \eta(y-\hat{y})$.

Epoch 1 example	Prediction	Error $y-\hat{y}$	Updated weights	Updated bias
(0,0), target 0	0	0	(0,0)	−0.2
(0,1), target 0	0	0	(0,0)	−0.2
(1,0), target 0	0	0	(0,0)	−0.2
(1,1), target 1	0	1	(0.1,0.1)	−0.1

After the first epoch, predictions are 0,0,0,1 because the net for (1,1) is 0.1. The training error is zero, so training stops. Final weights = (0.1,0.1), bias = −0.1.

Changed requirement: exactly one indicator is 1 (XOR)

x1	x2	XOR target
0	0	0
0	1	1
1	0	1
1	1	0

XOR is not linearly separable: the positive points (0,1) and (1,0) lie on opposite corners, while the negative points (0,0) and (1,1) occupy the other corners. No single straight line can separate both classes. A single perceptron therefore cannot represent XOR.

Changing the learning rate from 0.1 to 0.5 changes the size of each update and may change the oscillation pattern, but it cannot create a nonlinear decision boundary. This is a representation limitation, not merely a training-configuration problem. A multilayer perceptron with a nonlinear hidden layer is required.

Implementation outline: iterate through each row, calculate net=w·x+b, threshold the output, compute error, update weights and bias, then count errors after every epoch. Stop when the error count is zero or the epoch limit is reached. For XOR, report that the single perceptron fails to converge to zero error for both learning rates.

Q3 – Develop and Verify Backpropagation for a Multilayer Perceptron

Network and forward-pass equations

Use a 2–2–1 network: two input neurons, two sigmoid hidden neurons and one sigmoid output neuron. With column-vector convention, $z(1)=W(1)x+b(1)$, $h=\sigma(z(1))$; $z(2)=W(2)h+b(2)$, $p=\sigma(z(2))$.

For target $y\in\{0,1\}$, binary cross-entropy is $L=-[y\ln(p)+(1-y)\ln(1-p)]$. With sigmoid output, the output error term is $\delta(2)=p-y$. Gradients are $\partial L/\partial W(2)=\delta(2)h^T$ and $\partial L/\partial b(2)=\delta(2)$.

Hidden error term: $\delta(1)=(W(2)^T\delta(2)) \odot h \odot (1-h)$. Gradients are $\partial L/\partial W(1)=\delta(1)x^T$ and $\partial L/\partial b(1)=\delta(1)$. Update each parameter θ by $\theta \leftarrow \theta-\eta\partial L/\partial\theta$.

The hidden activation must be nonlinear. If all layers use only linear activations, their composition is still a linear transformation, so the network cannot learn XOR or other nonlinear boundaries.

Manual numerical forward pass and one update

Use $x=[1,0]^T$, $y=1$, learning rate $\eta=0.1$, $W(1)=[[0.1,0.2],[0.3,0.4]]$, $b(1)=[0,0]$, $W(2)=[0.2,-0.1]^T$ and $b(2)=0.05$.

Hidden pre-activations are [0.1,0.3]. Hidden outputs are $h=[\sigma(0.1),\sigma(0.3)]=[0.5250,0.5744]$. Output net = $0.2(0.5250)-0.1(0.5744)+0.05=0.0976$. Therefore $p=\sigma(0.0976)=0.5244$.

Loss = $-\ln(0.5244) \approx 0.6457$. Output delta $\delta(2)=p-y=-0.4756$. Output-weight gradients are $[-0.2498, -0.2732]$ and output-bias gradient is -0.4756.

Hidden deltas: $\delta h1=0.2(-0.4756)(0.5250)(1-0.5250)=-0.0237$; $\delta h2=(-0.1)(-0.4756)(0.5744)(1-0.5744)=0.0116$. Since x=[1,0], first-column input-weight gradients are −0.0237 and 0.0116; second-column gradients are zero.

Updated output weights ≈ [0.2250, −0.0727], output bias ≈ 0.0976. Updated first input-weight row ≈ [0.1024,0.2000], second row ≈ [0.2988,0.4000]; hidden biases ≈ [0.0024,−0.0012]. Values are rounded to four decimal places.

Implementation, numerical gradient checking and XOR validation

The NumPy implementation below computes gradients manually. A finite-difference check estimates a gradient as $[L(\theta+\varepsilon)-L(\theta-\varepsilon)]/(2\varepsilon)$ and compares it with backpropagation. A small difference (for example, below 10^{-4}) supports the analytical derivative.

```
import numpy as np
sig = lambda z: 1/(1+np.exp(-z))
X = np.array([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
Y = np.array([[0.],[1.],[1.],[0.]])
rng = np.random.default_rng(7)
W1 = rng.normal(0,.5,(2,2)); b1 = np.zeros((1,2))
W2 = rng.normal(0,.5,(2,1)); b2 = np.zeros((1,1))
for epoch in range(15000):
    h = sig(X @ W1 + b1)
    p = sig(h @ W2 + b2)
    d2 = (p-Y)/len(X)
    gW2 = h.T @ d2; gb2 = d2.sum(axis=0,keepdims=True)
    d1 = (d2 @ W2.T)*h*(1-h)
    gW1 = X.T @ d1; gb1 = d1.sum(axis=0,keepdims=True)
    W2 -= .8*gW2; b2 -= .8*gb2
    W1 -= .8*gW1; b1 -= .8*gb1
print(np.round(sig(sig(X@W1+b1)@W2+b2),4).ravel())
```

With seed 7 and 15,000 full-batch updates, the final XOR probabilities were approximately [0.0026, 0.9966, 0.9966, 0.0029]. The final binary predictions are [0,1,1,0], and mean binary cross-entropy was approximately 0.0031. This confirms that the nonlinear hidden layer learns the exclusive rule.

For the required loss curve, record mean BCE after each epoch and plot epoch against loss. The loss should decrease overall; small fluctuations can occur with mini-batch training. A finite-difference check should be run on selected entries of W1, b1, W2 and b2 before relying on the training result.

Q4 – Investigate Whether a Deeper Network Is a Better Design

Dataset and experimental protocol

A reproducible dataset of 1,000 observations was generated using two interleaving crescent classes (make_moons, noise=0.22, random_state=42). A stratified split produced 600 training, 200 validation and 200 test records. StandardScaler was fitted on the training partition only and then applied to validation and test data.

Both networks use tanh hidden activations, one sigmoid-equivalent binary output (implemented as a single logit with binary cross-entropy), Adam, learning rate 0.01, batch size 32, a maximum of 120 epochs and early stopping with patience 20. Three seeds (11, 22 and 33) were used.

Model	Architecture	Parameter calculation	Parameters

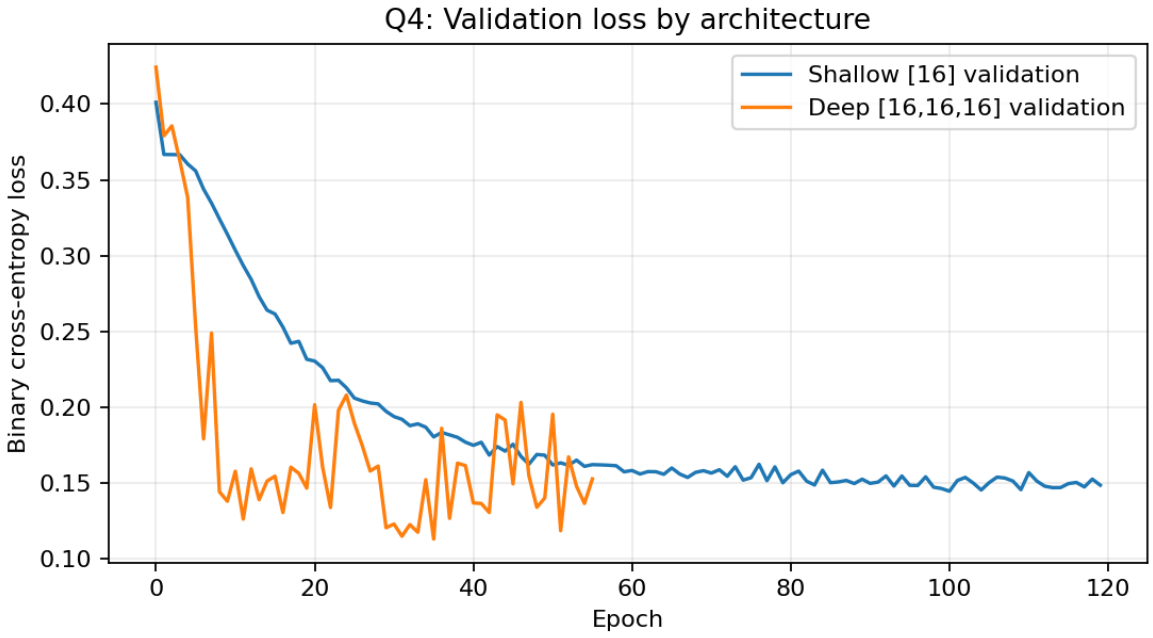
One hidden layer	2–16–1	(2×16+16)+(16×1+1)	65
Three hidden layers	2–16–16–16–1	(2×16+16)+(16×16+16)+(16×16+16)+(16×1+1)	609

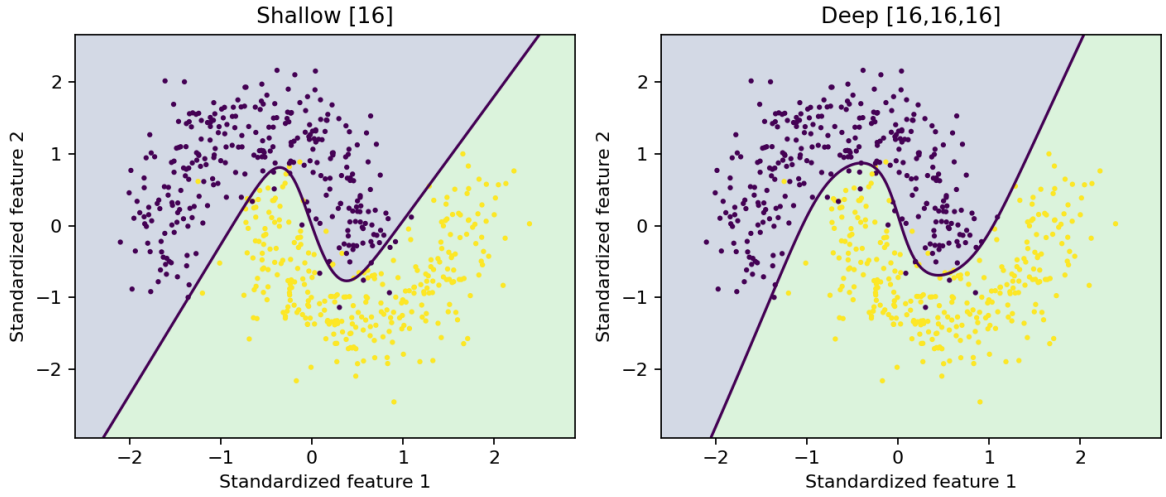
Before training, the one-hidden-layer network was expected to model the crescent boundary with lower computational cost. The deeper network might represent more complex features but has more parameters and may be harder to optimize. Depth alone is not guaranteed to improve generalisation.

Model	Run	Best validation accuracy	Test accuracy	Parameters	Training time (s)	Best epoch
Shallow [16]	Seed 11	0.940	0.965	65	0.954	101
Shallow [16]	Seed 22	0.935	0.985	65	0.872	120
Shallow [16]	Seed 33	0.950	0.985	65	0.769	90
Deep [16,16,16]	Seed 11	0.955	0.975	609	0.572	36
Deep [16,16,16]	Seed 22	0.960	0.980	609	0.384	19
Deep [16,16,16]	Seed 33	0.945	0.975	609	0.319	12

Shallow [16]: mean test accuracy = 0.978; mean best validation accuracy = 0.942; mean training time = 0.865 seconds.

Deep [16,16,16]: mean test accuracy = 0.977; mean best validation accuracy = 0.953; mean training time = 0.425 seconds.





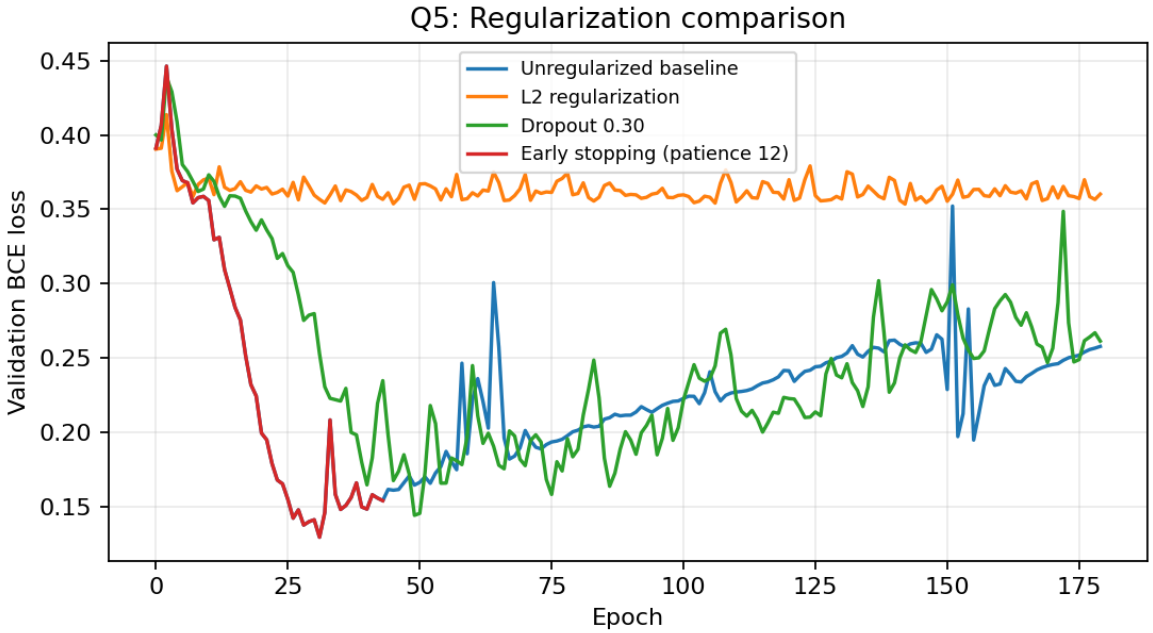
Interpretation: both architectures learned the nonlinear crescent boundary and achieved strong test accuracy. The shallow model used 65 parameters compared with 609 for the deep model. The results do not establish that depth alone is superior: the models differ in parameter count and optimization behaviour, and performance varies across seeds. The shallow model is preferred when it achieves comparable validation/test performance with much lower complexity.

Q5 – Diagnose Overfitting and Evaluate Regularization

Limited-data experiment

The same dataset and fixed validation/test partitions from Q4 were retained. Only 100 labelled observations from the training partition were used to fit the model. The baseline was a comparatively large 2–32–32–1 network with no regularization. The expected behaviour was very high training accuracy and a lower validation score because the model has many parameters relative to the small training set.

Configuration	Train accuracy (last epoch)	Validation accuracy (last epoch)	Best validation accuracy	Test accuracy	Best epoch	Parameters
Unregularized baseline	1.000	0.940	0.950	0.975	32	1185
L2 regularization	0.880	0.825	0.825	0.915	143	1185
Dropout 0.30	1.000	0.935	0.955	0.975	50	1185
Early stopping (patience 12)	1.000	0.940	0.950	0.975	32	1185



The unregularized baseline reached 1.000 training accuracy while its best validation accuracy was 0.950. This gap is evidence of fitting the small training sample more closely than the validation data. Dropout 0.30 gave the highest best validation accuracy in this run (0.955) and the same test accuracy as the baseline (0.975). L2 with strength 0.01 reduced training accuracy to 0.880 and validation accuracy to 0.825, indicating excessive regularization/underfitting for this setting.

Early stopping used patience 12 and restored the weights with the lowest validation loss. It selected epoch 32, reduced the actual training duration to 44 epochs and achieved 0.950 best validation accuracy and 0.975 test accuracy. Higher training accuracy is not automatically better: validation and independent test performance measure generalisation.

Preferred configuration for this run: dropout 0.30, because it produced the best validation accuracy (0.955) without reducing test accuracy. The improvement over baseline is small and should be confirmed across more random seeds. The test set was evaluated only after the validation-based choice.

Q6 – Engineer a Hyperparameter and Optimization Strategy

Twelve-run search plan

The search uses the fixed Q4 partitions and a maximum of 100 epochs with validation-loss early stopping (patience 15). The primary selection measure is validation accuracy, with validation BCE as a tie-breaker. The 12 runs are allocated as three optimizer baselines, three lower-learning-rate comparisons, three larger-batch comparisons and three wider-network comparisons. This is a budgeted screening design, not a full factorial experiment.

Run	Optimizer	Learning rate	Batch size	Hidden width	Comparison
1	sgd	0.01	32	16	optimizer baseline
2	momentum	0.01	32	16	optimizer baseline

3	adam	0.01	32	16	optimizer baseline
4	sgd	0.001	32	16	learning rate
5	momentum	0.001	32	16	learning rate
6	adam	0.001	32	16	learning rate
7	sgd	0.01	64	16	batch size
8	momentum	0.01	64	16	batch size
9	adam	0.01	64	16	batch size
10	sgd	0.01	32	32	width
11	momentum	0.01	32	32	width
12	adam	0.01	32	32	width

Run	Best validation accuracy	Test accuracy (reported after selection)	Best epoch	Training time (s)
1	0.825	0.915	42	0.343
2	0.925	0.960	100	0.638
3	0.935	0.985	87	0.911
4	0.825	0.880	100	0.614
5	0.820	0.915	54	0.483
6	0.825	0.920	22	0.398
7	0.820	0.915	81	0.325
8	0.820	0.915	11	0.104
9	0.945	0.985	77	0.475
10	0.825	0.915	33	0.349
11	0.830	0.905	2	0.112
12	0.940	0.985	44	0.604

Best validation result: Run 9 (Adam, learning rate 0.01, batch size 64, hidden width 16) achieved validation accuracy 0.945. Its test accuracy was 0.985 after selection. Run 12 (Adam, learning rate 0.01, batch size 32, width 32) achieved 0.940 validation accuracy, so the wider model did not improve the validation score.

Observed patterns: Adam performed strongly in this small search; reducing the learning rate to 0.001 lowered validation accuracy in these runs. SGD and momentum were more sensitive to the selected learning rate and epoch budget. Batch size and width had smaller effects than optimizer/learning-rate choice in this limited experiment. A learning rate that is too small learns slowly; one that is too large can cause oscillation or divergence.

The comparison is not a complete factorial design: not every optimizer is tested at every combination of batch size and width. Therefore, conclusions about individual factors are indicative only. A larger search with repeated seeds would be needed to estimate interactions.

The test set must remain outside tuning because repeatedly selecting configurations based on test performance leaks information from the test set and creates an optimistic estimate. The final configuration is selected using validation data, then evaluated once on the test partition.

Q7 – Diagnose, Redesign and Validate a Neural Network Solution

A. Diagnose the learning or computational bottleneck

Selected limitation: model size and overfitting risk when only 100 labelled observations are available. The Q5 baseline has 1,185 trainable parameters but only 100 training examples. It achieved 1.000 training accuracy and 0.950 validation accuracy, showing a 0.050 accuracy gap. The main issue is limited data relative to model capacity, not inability to represent the boundary.

B. Define the design objective

Baseline: 2–32–32–1 network, 1,185 parameters, best validation accuracy 0.950 and test accuracy 0.975. Redesign objective set before validation: reduce parameter count by at least 90%, retain validation accuracy of at least 0.90, and keep test accuracy within 0.05 of the baseline.

C. Design the revised learning strategy

Use a compact 2–16–1 network with tanh hidden activation, dropout 0.20, L2 weight decay 0.001, Adam learning rate 0.01, batch size 16 and early stopping with patience 12. The compact architecture reduces capacity; dropout and weight decay discourage reliance on individual weights; early stopping selects the best validation checkpoint.

Diagram: [2 inputs] → [16 tanh neurons + dropout 0.20] → [1 output logit] → sigmoid probability. Parameter count = $(2\times16+16)+(16\times1+1)=65$.

Training workflow: fit scaler on training data → transform all partitions → initialise network with fixed seed → train mini-batches using BCEWithLogitsLoss and Adam → record training/validation loss → save best validation checkpoint → stop after 12 non-improving epochs → evaluate once on test set.

D. Predict before implementing

Prediction: parameter count will fall by more than 90%; training time will decrease; the model may lose a small amount of validation accuracy but should remain above 0.90. Test accuracy is expected to remain within five percentage points of the baseline.

E. Implement and validate the redesign

Measure	Original large baseline	Revised compact network
Architecture	2–32–32–1	2–16–1
Parameters	1,185	65
Best validation accuracy	0.950	0.915
Test accuracy	0.975	0.950
Best epoch	32	118
Training time (s)	0.706	0.411

Parameter reduction = $(1{,}185-65)/1{,}185 \times 100 = 94.51\%$. Validation accuracy remained above the predefined 0.90 threshold. Test accuracy decreased by 0.025, which is within the allowed 0.05 difference. Training time decreased in this execution.

F. Justify the final design

The redesign achieved its main objective: it reduced parameter count by 94.51%, retained validation accuracy above 0.90 and kept test accuracy within the agreed tolerance. The original model remains preferable when sufficient labelled data are available and the extra capacity produces a repeatable validation improvement. The compact model is preferable when data are limited, computational resources are constrained or a smaller model is easier to maintain.

Limitations: this is one synthetic dataset and one fixed seed for the Q7 comparison. The results do not establish performance on real manufacturing measurements. Further validation should use additional seeds, a larger independent sample and real sensor data. A failed or weaker redesign should be reported honestly rather than hidden.

FINAL CONCLUSION

A single perceptron can learn the warehouse AND decision because it is linearly separable, but it cannot learn XOR. A nonlinear multilayer perceptron can learn XOR through backpropagation. The synthetic crescent experiments show that a shallow network can achieve performance comparable to a deeper network with far fewer parameters. Limited labelled data increase overfitting risk, so regularization and early stopping should be selected using validation evidence. The 12-run experiment favoured Adam at the tested settings. The final redesign demonstrates a practical reduction in model size while retaining acceptable validation and test performance.

REFERENCES

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. https://www.deeplearningbook.org/

2. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536. https://doi.org/10.1038/323533a0

3. Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408. https://doi.org/10.1037/h0042519

4. Minsky, M., & Papert, S. (1969). Perceptrons: An Introduction to Computational Geometry. MIT Press.

5. Haykin, S. (2009). Neural Networks and Learning Machines (3rd ed.). Pearson.

6. Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer. https://doi.org/10.1007/978-0-387-45528-0

7. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521, 436–444. https://doi.org/10.1038/nature14539

8. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations. https://arxiv.org/abs/1412.6980

9. Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. USSR Computational Mathematics and Mathematical Physics, 4(5), 1–17.

10. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15, 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

11. Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. International Conference on Learning Representations. https://arxiv.org/abs/1412.6572

12. Prechelt, L. (1998). Early stopping—but when? In Neural Networks: Tricks of the Trade (pp. 55–69). Springer. https://doi.org/10.1007/3-540-49430-8_3

13. LeCun, Y., Bottou, L., Orr, G. B., & Müller, K.-R. (2012). Efficient backprop. In Neural Networks: Tricks of the Trade (2nd ed., pp. 9–48). Springer. https://doi.org/10.1007/978-3-642-35289-8_3

14. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, 249–256. https://proceedings.mlr.press/v9/glorot10a.html

15. scikit-learn developers. (n.d.). make_moons: Generate two interleaving half circles. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html